Case study

ART·E: an email agent trained with RL

How OpenPipe built an RL environment for inbox search, and trained a 14B model to beat o3 on it

What you will learn from this case

1. How to make realistic tasks from real data with an LLM.
2. How to give an agent a small, safe set of tools.
3. How to design a reward that does not invite hacks.
4. How to spot a reward hack, and how to fix it.
5. How one RL environment serves training and evaluation.

The problem: answer questions from an inbox

> A user asks a question in plain words. The agent searches the inbox, reads emails, and answers with its sources.

Why it is hard: the answer hides in one email among thousands.

Why RL helps: a model learns better search habits by practice, not by rules we write.

Why it is a good case: the answer is checkable against a reference.

The RL environment at a glance

Part	In ART·E
Task set	About 4,000 synthetic questions on real Enron inboxes
Harness	Three tools: search, read, answer. At most 10 turns.
State	A read-only SQLite database of emails
Grader	An LLM judge compares the answer with a reference
Split	20 inboxes for training, 8 other inboxes for testing

How the tasks were made

1. Take real inboxes from the Enron corpus, about 500,000 emails.
2. Give GPT-4.1 a batch of 20 emails from one inbox.
3. It writes a question, the answer, and the ID of the source email.
4. It also scores how realistic the question is, from 0 to 1.
5. Drop questions with a low realism score.

Result: about 4,000 question and answer pairs.

One task, as data

```
question: str
answer: str
message_ids: list[str]
how_realistic: float
inbox_address: str
query_date: str
split: "train" | "test"
```

The query date matters: search sees only emails sent before it, so the model cannot read the future.

The harness: three tools and a short loop

```
search_inbox(keywords) → up to 10 IDs + snippets
read_email(message_id) → the full email
return_final_answer(answer, source_ids)
```

Search: SQLite full-text search (FTS5), only in the user's own inbox.

Loop: model, then tool, then result. Stop at a final answer or after 10 turns.

The reward: correctness first

Outcome of one rollout	Reward
Correct answer, few turns	**Highest**
Correct answer, more turns	**High**
“I don’t know”	**Low**
Wrong answer	**Lowest: a large penalty**

OpenPipe did not publish exact values. For GRPO, the order matters most.

The grader: an LLM judge with a reference

> “Accept the answer if it contains the relevant information from the reference answer.”
>
> “Do not accept the answer if it is missing information relevant to the question, or if it contradicts the reference answer.”

Input: the question, the reference answer, the AI answer.

```
Output: reasoning: str, accept: bool
```

What broke, and how they caught it

The hack

An early reward gave credit for extra turns. The model repeated its last tool call until it hit the turn limit.

How they caught it

They read rollouts by hand, not only the reward curve.

The fix

Remove the turn credit. Keep the reward on the final answer.

The training setup

Setting	Value
Base model	Qwen2.5 14B
Algorithm	GRPO with the open ART library
Each step	12 questions × 4 rollouts = 48 rollouts
Learning rate	1.2e-5, for 2 epochs
Validation	Every 30 steps, on 100 questions
Compute	One H100 GPU, under a day, about $80

Results: answer accuracy

~96%

ART·E, a trained 14B model

~90%

o3, the best prompted model

60%

fewer errors: 10% down to 4%

ART·E also made up fewer answers than every prompted model they tested.

This is one narrow task. It does not show that the small model beats o3 in general.

Results: cost to run 1,000 searches

~$55
o3

~$8
o4-mini

~10×
less than o4-mini: ART·E

Building it cost about $80 of GPU time and about one week of an expert’s work.

One RL environment, two jobs

Training

Questions from the 20 training inboxes. Four rollouts each. Rewards go to GRPO.

Evaluation

Questions from the 8 test inboxes. Same tools, same judge. Scores compare models.

Lesson

Only the inboxes and the use of the score change. So the o3 baseline and the trained model face the same test.

A newer reward: RULER

How it works	An LLM judge sees all rollouts for one question together. It scores each from 0 to 1.
Why it fits	GRPO needs only the order inside a group. Ranking is easier than absolute scoring.
In ART·E today	RULER gives the reward. The reference-answer judge still runs, as a metric.

You can train without writing a reward function. You still need a check you trust.

Six design rules from ART·E

1. Run a prompted baseline first. It finds bugs and sets a target.
2. Split the data by inbox, not by question.
3. Let correctness dominate. Keep extra rewards small.
4. Make “I don’t know” better than a wrong answer.
5. Read rollouts, not only reward curves.
6. Watch reward spread in each group. No spread, no learning.

Try it yourself

Notebook	art-e.ipynb in OpenPipe’s art-notebooks repository, on Colab.
Data	Loads from Hugging Face. It builds the SQLite email database for you.
Keys	A Weights and Biases key for training. An OpenAI key for the judge.
Compute	Training runs on a hosted backend. You need no local GPU.

```
pip install openpipe-art
```

An Indian version: a Hinglish inbox

Inboxes	Synthetic: train tickets, UPI receipts, college notices, family chats.
Questions	In English, Hindi and Hinglish, with a realism score.
New challenge	Keyword search across scripts: “ticket” versus “टिकट”.
Research question	Does training on Hinglish questions help English ones?

Use only synthetic personal data. Never use a real inbox.

Questions for discussion

1. The question writer saw the answer email. How could that make the tasks too easy?
2. How could a model fool the LLM judge? How would you test for it?
3. What should the reward do if the answer is right but the sources are wrong?
4. Is 96% against 90% on one task enough to choose a trained model?

Read the originals

- Kyle Corbitt’s talk: How to Train Your Agent (2025)
- ZenML: Building ART·E
- Machine Learning at Scale: OpenPipe RL for multi-turn agents
- The ART·E notebook
- The ART library on GitHub
- RULER documentation